

Assignment 2

Introduction to Parallel Programming

due **Friday 25 September 2026, 23:59** (hard deadline)

Instructions

Although it is allowed to work alone, we suggest that you try to find another student to work together. (You will be allowed to change groups, if you want, in the next assignments.) In both cases, you must register in a group in Studium under People/Lab Groups before submission. In case of trouble, contact the assistant (xiaoyue.chen@it.uu.se) responsible for this assignment.

Note **that this is the assignment which requires the most time/effort in this course**, so start early and plan well.

Submission checklist:

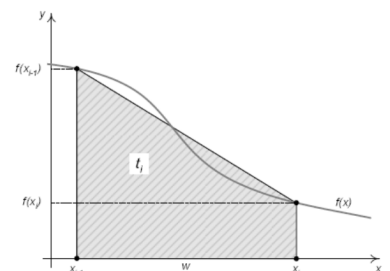
- Submissions must clearly show your name(s).
- Submit a **single** PDF report, as well as all source code related to the exercises on Studium.
- Solutions must be in C11/C++11 (or newer), as specified in the exercises.
- Use appropriate synchronization mechanisms from the C or C++ standard libraries. (For the C++ one refer to <http://en.cppreference.com/w/cpp/thread>.) Do **not** rely on `semaphore.h`, `sem_t`, etc.
- All source code must compile and run on the **Linux lab machines running Ubuntu**. For the list of the Linux servers that you can use, refer to <https://www.it.uu.se/datordrift/maskinpark/linux>.
- **Provide instructions for compilation and running.**
- No source code modifications should be required for reproducing your results.
- Your report must describe the theoretical concepts used as well as all relevant details of your solution.
- When reporting time, performance or scalability, use the appropriate speedup and timing plots and make sure they are as reader-friendly as possible.

Recall that you need to score half the points (**seven** in this assignment) or more to pass an assignment. In case you do not reach a working solution, describe the main challenges and proposals to address them. Please keep your answers short and concise, but clear and complete.

Also, note that some of the exercises ask you to do benchmarking on the Lab machines and this requires 1) using the machine at a time when it is lightly loaded, and 2) taking multiple measurements to see if there is any variation. Thus, it's not a good idea to leave this part of the assignment for too close to the deadline!

Exercise 1: Numerical Integration (3 points)

Numerical integration is a technique for numerical (approximate) calculation of a definite integral of a function f . The algorithm is defined as follows: The interval of the integral is partitioned into many small parts of size w . For each of those small intervals the area under the curve is approximated by the area of the trapeze defined by x_{j-1} , x_j , $f(x_{j-1})$ and $f(x_j)$. Then the approximation of the whole integral can



be computed by summing up all trapeze areas. The total accuracy of the calculation can be scaled arbitrarily by increasing the total number of trapezes. As the calculation of one trapeze area is independent from the other areas, this parallel numerical integration algorithm can be parallelized quite easily.

- a) Implement a Posix threads program in C11/C++11 (or newer) for numerical integration of the following function:

$$\int_0^1 \frac{4}{1+x^2} dx$$

Your program should accept the total number of threads and trapezes with appropriate command-line arguments. Also, the command line argument `-h` should print a help message and exit.

- b) Test your program and measure the runtime of the computation (i.e., without setup time) for various configurations (number of threads, number of trapezes, each starting at 1). However, make sure that you also measure for a large number of trapezes (in the order of tens or hundreds of thousands).

Document your results in tabular form and analyze. By the way, did you notice something concerning the value of the computed integral?

- c) Evaluate and document different methods of distributing the work among threads.

Hint: you may consider using different numbers of threads and trapezes.

You need to provide: 1) your code and 2) your report. Each is worth 1.5 points.

Exercise 2: Sieve of Eratosthenes (3 points)

For this exercise, you will use Posix threads to implement a parallel version of the Sieve of Eratosthenes,¹ an algorithm to find prime numbers. For some maximum positive integer Max , the Sieve of Eratosthenes works as follows:

1. Create a list of natural numbers: $1, 2, 3, \dots, Max$.
2. Set k to 2, the first unmarked number in the list.
3. repeat

Mark all multiples of k between k^2 and Max .

Find the smallest number greater than k that is still unmarked.

Set k to this new value.

until k^2 is greater than Max .

4. The unmarked numbers are all prime.

To parallelize this algorithm:

1. First sequentially compute primes up to $\sqrt{Max}$.
2. Given p cores, build p chunks of roughly equal length covering the range from $\sqrt{Max} + 1$ to Max , and allocate a thread for each chunk.
3. Each thread uses the sequentially computed “seeds” to mark the numbers in its chunk.
4. The master waits for all threads to finish and collects the unmarked numbers.

¹<https://www.famousscientists.org/eratosthenes/>

Besides your code, you need to provide a brief report that explains your solution (e.g., did you use synchronization between threads and why, how did you distribute the work to threads, how did you minimize communication, how did you achieve load balance, etc.) and reports the speedup curve you get as the number of cores is increased. You should also try to vary the size of *Max* to gauge the impact of program size on parallel performance. Make sure to pick large numbers for *Max* (e.g., in the millions to ensure there's enough work for threads to do). Needless to mention, make sure your code is correct.

Exercise 3: Mutual Exclusion Algorithms (3 points in total)

Programmers at the Flaky Computer Corporation designed the protocol shown below to achieve n-thread mutual exclusion.

```
class Flaky implements Lock {
    private int turn;
    private boolean busy = false;
    public void lock () {
        int me = ThreadID.get();
        do {
            do {
                turn = me;
            } while (busy);
            busy = true;
        } while (turn != me);
    }
    public void unlock() {
        busy = false;
    }
}
```

For each question, either sketch a proof or display an execution where it fails.

1. Does this protocol satisfy mutual exclusion?
2. Is this protocol starvation-free?
3. Is this protocol deadlock-free?

Exercise 4: Concurrent Data Structure (5 points in total)

Concurrent data structures require proper synchronization so that users of the data structure do not have to think about it themselves. The example given in this exercise is a sorted singly linked list data structure with the following interface:

insert inserts a specified value into the linked list at the correct position.

remove removes one copy of the specified value if it is in the list.

count counts the number of elements with the specified value.

You can find a sequential implementation of this data structure in `sorted_list.hpp`. The driver program in `benchmark_example.cpp` prefills the sorted list with a number of elements and then runs three benchmarks:

- a read-only benchmark, using only `counts`.
- a write-only benchmark, using 50% `inserts` and 50% `removes`.

```
$ g++ -Wall -std=c++17 -pthread -O3 benchmark_example.cpp -o bench
$ ./bench 1
non-thread-safe read / threads: 1 - thousands of operations per second: 84937.607182
non-thread-safe update / threads: 1 - thousands of operations per second: 55.415223
non-thread-safe mixed / threads: 1 - thousands of operations per second: 779.892258

$ ./bench 2
non-thread-safe read / threads: 2 - thousands of operations per second: 169855.388171
^C
$
```

Figure 1: Benchmarking the driver program of Exercise 4.

- a mixed benchmark, using 3.125% inserts, 3.125% removes, and 93.75% counts.

You can test the driver program as shown in Figure 1. In the second run, the program stopped progressing and we had to abort it manually. As the program contains data races, it may also randomly crash, delete random files, or spawn demons from your nose.² Fortunately, all it does in this case is to hang.

²See <http://www.catb.org/jargon/html/N/nasal-demons.html>.

Your task is to implement (in C++) and compare thread-safe versions of the sorted list, using the following synchronization mechanisms:

1. coarse-grained locking (for example, in C++, using the `std::mutex` class from the standard library; see <http://en.cppreference.com/w/cpp/thread/mutex>);
2. fine-grained locking (again, using the `std::mutex` class from the C++ standard library);
3. coarse-grained locking, using a *test-and-test-and-set* (TATAS) lock implementation;
4. fine-grained locking, using a *test-and-test-and-set* (TATAS) lock implementation;
5. fine-grained locking, using a *scalable queue lock* algorithm, either CLH or MCS—feel free to choose between them.

Using the standard library's mutex locks should be straightforward. For your own lock implementation, we suggest to use the standard C++ library atomic functions and classes (e.g., `std::atomic<bool>`; see <http://en.cppreference.com/w/cpp/atomic/atomic>).

Evaluate, plot, and explain the results/behaviour of the five versions from the point of view of performance and scalability with respect to:

- the number of threads;
- the performed operations.

Make sure you run your experiments on a machine that allows to run at least 16 concurrent threads (e.g., one that has at least 8 physical cores, each with hyperthreading), and run your experiments using a suitable subset of worker threads in that range, making sure you include all powers of two ($1, 2, \dots, 2^n$) in the range of numbers that your machine allows as concurrent threads.

Discuss the advantages and disadvantages of each form of synchronization in terms of: ease of implementation, lock contention, performance, and scalability. Identify situations when one version is more suitable than another, e.g. % of read vs. % of update operations.

Dedicate a section of your report to explain the reasoning, challenges and solutions in implementing the five versions of the data structure. Describe both the general solution and technical details. If you ended up getting 'inspiration' from code for some of these locks that you found on the internet, make sure you include a pointer to it in your source and your report.

The points of this exercise are split equally into 2.5 points for implementation and 2.5 for the report.

Good luck!